



南開大學
Nankai University

高级语言程序设计 实验报告

南开大学 计算机大类

姓名：赵宇哲

学号：2411094

班级：模拟 3-3 班

2025 年 3 月 22 日

目录

1 作业题目	2
2 开发软件	2
3 课题要求	2
4 主要流程	3
4.1 整体概述	3
4.2 基础类的实现	4
4.3 光线追踪的原理	5
4.4 核心函数的实现	6
4.4.1 光线碰撞	6
4.4.2 阴影效果	6
4.4.3 2 * 2 多重采样抗锯齿 (MSAA)	8
4.4.4 光线追踪	10
4.5 phong 模型的引入	12
4.6 窗口的绘制及 UI 设计	12
5 单元测试	13
6 收获	14

1 作业题目

基于 phong 模型的 Ray Tracing 渲染器

2 开发软件

Qt Creator 15.0.1 (Community)

3 课题要求

1. 面向对象。
2. 单元测试。
3. 模型部分。
4. 验证。

4 主要流程

4.1 整体概述

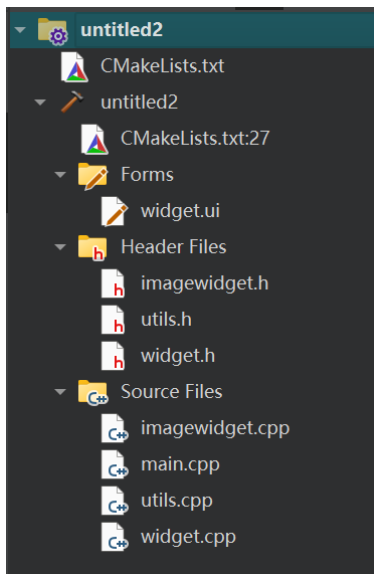


图 4.1.1

Forms 文件夹

这个文件夹包含 Qt Designer 生成的 UI 文件 (.ui)。定义了应用程序的用户界面布局和控件。

- **widget.ui**: 这是主窗口的用户界面文件，本项目使用了 Qt Designer 设计并并在相应的 cpp 文件中实现了信息传递。

Header Files 文件夹

这个文件夹包含了项目的头文件 (.h)，定义了类的声明和函数的原型。

- **imagewidget.h**: 定义了 **ImageWidget** 类，用于绘制渲染结果。
- **utils.h**: 定义了辅助函数和类，如向量类 **Vector3**、球体类 **Sphere**、光源类 **Light** 等。**这些类是本项目的核心。**
- **widget.h**: 定义了主窗口类 **Widget**，负责初始化用户界面和处理用户交互。

Source Files 文件夹

这个文件夹包含了项目的源文件 (.cpp)，实现了头文件中声明的类和函数。

- **imagewidget.cpp**: 实现了 **ImageWidget** 类的构造函数和绘制逻辑。
- **main.cpp**: 程序的入口文件，初始化 Qt 应用程序和主窗口。
- **utils.cpp**: 实现了 **utils.h** 中声明的辅助函数和类的方法。
- **widget.cpp**: 实现了 **Widget** 类的构造函数和用户交互逻辑。

4.2 基础类的实现

向量类 (Vector3)

这是后续光照计算的基础。用于表示三维空间中的点、方向或颜色等。包含基本的向量运算，如加法、减法、数乘、点积、模长计算和单位化等。

球体类 (Sphere)

表示场景中的一个球体对象。包含球体的中心点、半径、颜色、材质类型、是否有纹理、纹理颜色和反射率等属性。

光源类 (Light)

表示场景中的光源。包含光源的位置、颜色和强度等属性。

光线类 (Ray)

表示从一点出发沿着特定方向的光线。包含光线的起点、方向和参数 t （单位向量的长度）。由 $A + tB$ ，我们可以表达出任意一条光线向量。

4.3 光线追踪的原理

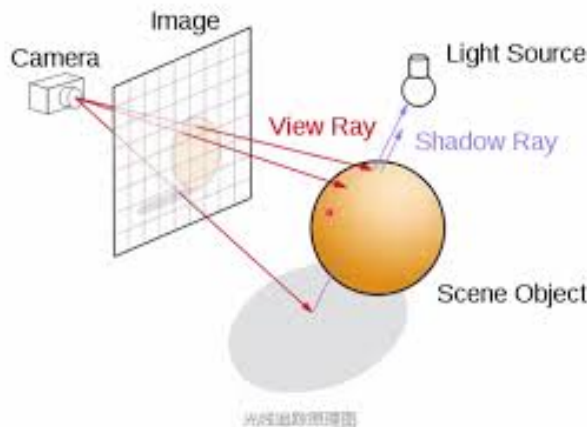


图 4.3.1

光线追踪的基本原理是光路可逆。这意味着光线从光源发出，经过物体反射或折射后进入人眼，形成图像。在光线追踪中，我们可以假设由人眼（camera）发出光线，对每个像素点的光线信息进行计算并返回相应值，从而得到渲染效果图片。

具体步骤如下：

1. 确定相机的位置。
2. 对于图像中的每个像素点，从相机位置沿着该像素点的方向发射一条光线。
3. 计算这条光线与场景中物体的交点。
4. 根据交点处的光照情况（不同材质具有不同的漫反射率以及不同光源具有不同光色和光强等）计算光线的信息。
5. 将计算得到的信息赋给对应的像素点。

但过程中会涉及到特殊情况，如：

光线与物体的碰撞

光线与物体的碰撞会产生反射现象，这是光线追踪中的关键步骤。它涉及到光线与物体表面的相互作用，包括漫反射、镜面反射等。这些反射现象会影响光线的颜色和方向，从而影响最终的渲染效果。

阴影效果

光线追踪还会计算阴影效果。当一条光线被物体遮挡时，它无法直接到达观察者，从而在观察者眼中形成阴影。阴影效果的计算涉及到光线与物体的相对位置和形状，以及光源的位置和强度。

这些光线追踪的原理和现象在 `utils.cpp` 中进行了对应的函数实现来进行处理。我们会在后续的 4.4 核心函数的实现部分进行详细讲解。

4.4 核心函数的实现

4.4.1 光线碰撞

在光线追踪算法中，光线碰撞检测是确定光线与场景中物体交互的关键步骤。这一过程涉及两个主要函数：`if_Hit` 和 `where_Hit`。

1. `if_Hit` 函数用于检查光线是否与球体相交。该函数通过计算光线方程和球体方程的交点，判断光线是否与球体相交。（原理是根据点线距离公式解一元二次方程）
2. `where_Hit` 函数用于计算光线与球体相交的具体位置。如果 `if_Hit` 函数确定光线与球体相交，`where_Hit` 函数将计算交点的确切位置。

问题及解决

问题：当球体设置到视野之外时，可能会被错误显示。

解决方式：忽视了 $t < 0$ 的情况（即光线反向延伸了）

```
if(t<=0&&(x1>x2?x1:x2)>0)
{
    t=(x1>x2?x1:x2);
}
```

图 4.4.1: 修正

4.4.2 阴影效果

阴影的产生是由于光线在传播过程中被物体阻挡，导致某些区域无法直接接收到光源的照射。在光线追踪中，我们可以通过以下步骤计算阴影效果：

1. 确定光源的位置。
2. 对于场景中的每个物体，计算从物体表面点到光源的直线路径。
3. 检查这条路径上是否有其他物体阻挡。如果有物体阻挡，则该点处于阴影中。
4. 根据物体是否处于阴影中，调整光照计算，减少或消除被阴影覆盖区域的光照。

在光线追踪算法中，通常使用射线投射法来计算阴影：

1. 对于每个需要计算阴影的点（例如，从相机发出的光线与物体相交的点），从该点向光源方向发射一条射线。
2. 检查这条射线是否与场景中的其他物体相交。这可以通过遍历所有物体，并使用光线碰撞检测算法（如 `if_Hit` 和 `where_Hit` 函数）来实现。
3. 如果射线与任何物体相交（且交点在从光源到计算点的路径上），则该点处于阴影中。
4. 如果射线没有与任何物体相交，则该点不在阴影中，可以接收到完整的光照。

效果展示

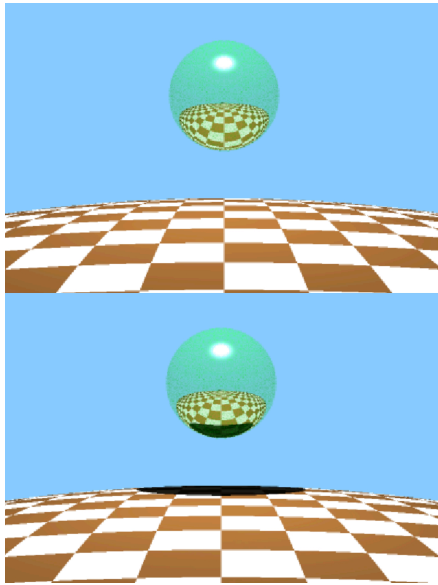


图 4.4.2: 添加阴影检测前后对比

代码实现

以下是计算阴影效果的代码实现：

```
1 bool isInShadow(const Vector3 &hitPoint, const Light &light, const
  std::vector<Sphere> &spheres, bool isReflection) {
2   Vector3 lightDir = Vector3::unitization(light.position - hitPoint);
3   Ray shadowRay(hitPoint + lightDir * 1e-3, lightDir, 0);
4   for (const auto& sphere : spheres) {
5       // 排除自身球体的阴影检测（避免误判自身导致阴影）
6       Vector3 t = hitPoint - lightDir * 1e-3;
7       if (sphere.center.x == t.x && sphere.center.y == t.y && sphere.center.z ==
          t.z) continue;
8
9       // 若为反射追踪且是地面球，跳过阴影检测
10      if (isReflection && sphere.materialType == FABRIC) continue;
11
12      if (if_Hit(sphere.center, sphere.radius, shadowRay)) {
13          return true;
14      }
15  }
16  return false;
17 }
```

这段代码通过向光源发射射线，并检查射线是否与其他物体相交，从而确定交点是否处于阴影中。

4.4.3 2 * 2 多重采样抗锯齿 (MSAA)

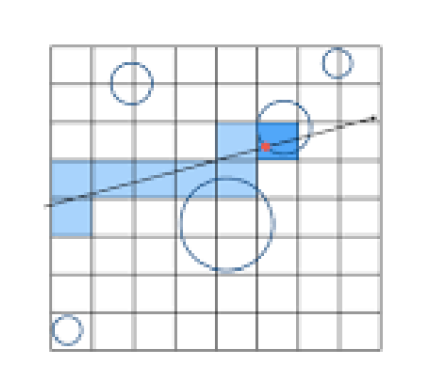


图 4.4.3: 简易原理图例

抗锯齿 (MSAA) 原理主要包括以下几个步骤:

1. 在每个像素内, MSAA 会在周围 4 个子像素 (与具体采样次数有关, 这里以 $2 * 2$ 为例) 位置进行采样。
2. 对于每个采样点, MSAA 会收集所有子像素的颜色值, 并计算它们的平均值, 以确定最终的像素颜色。
3. MSAA 还会对深度值进行多重采样, 以确保在深度测试中也能获得更好的效果。

注: 该方法涉及原有坐标与 “世界坐标” 之间的转化

转化方法可理解为求解反函数, 即: 已知 $y = f(x)$, 借此求出 $x = g(y)$ 的表达式

效果展示

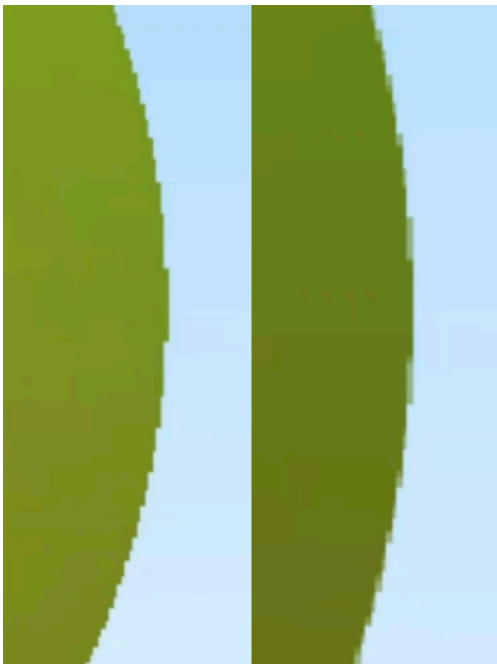


图 4.4.4: 添加抗锯齿前后前后对比

代码实现

以下是实现多重采样抗锯齿的代码：

```
1 Vector3 multisample(double x, double y, const std::vector<Sphere> &spheres, const
  Light &light) {
2     const int sample = 2;
3     Vector3 sample_color[sample * sample];
4     int n = 0;
5
6     for (int i = 0; i < sample; i++) {
7         for (int j = 0; j < sample; j++) {
8             double newx = x - 0.5 + (i + 0.5) / sample;
9             double newy = y - 0.5 + (j + 0.5) / sample;
10            Ray ray(camera, Vector3(newx, newy, z), 0);
11            sample_color[n++] = traceRay(ray, spheres, light);
12        }
13    }
14
15    Vector3 average_color = sample_color[0];
16    for (int i = 1; i < sample * sample; i++) {
17        average_color = average_color + sample_color[i];
18    }
19    average_color = average_color / (sample * sample);
20
21    average_color.x = std::clamp(average_color.x, 0.0, 255.0);
22    average_color.y = std::clamp(average_color.y, 0.0, 255.0);
23    average_color.z = std::clamp(average_color.z, 0.0, 255.0);
24
25    return average_color;
26 }
```

这段代码通过在每个像素内进行多次采样，并计算平均颜色值，从而实现抗锯齿效果。这种方法可以有效地减少锯齿现象，提高渲染图像的质量。

4.4.4 光线追踪

`traceRay` 函数是该项目的核心。它负责模拟光线在场景中的传播和与物体表面的交互，包括反射和折射。下面我将详细讲解该函数：

递归深度检查

```
1 const int MAX_DEPTH = 3;
2 if (depth > MAX_DEPTH) return Vector3(0,0,0);
```

这里定义了光线追踪的最大递归深度为 3。如果超过这个深度，函数将返回黑色(`Vector3(0,0,0)`)，表示光线没有更多的反射或折射（从物理学上来理解可以认为是光的能量已经不足以让人眼识别）。

初始化变量

```
1 double closestDist = std::numeric\_limits<double>::max();
2 const Sphere *closestSphere = nullptr;
3 Vector3 hitPoint;
4 Vector3 normal;
```

初始化一些变量，用于存储最近的交点信息。`closestDist` 用于存储最近的交点距离，`closestSphere` 用于存储与光线相交的球体，`hitPoint` 和 `normal` 用于存储交点位置和法线方向。

寻找最近的交点

```
1 for (const auto &sphere : spheres) {
2     if (if\_Hit(sphere.center, sphere.radius, ray)) {
3         Vector3 currentHit = where\_Hit(sphere.center, sphere.radius, ray);
4         double dist = Vector3::getlen(currentHit - ray.A);
5         if (dist < closestDist) {
6             closestDist = dist;
7             closestSphere = &sphere;
8             hitPoint = currentHit;
9             normal = Vector3::unitization(hitPoint - closestSphere->center);
10        }
11    }
12 }
```

遍历所有球体，使用 `if_Hit` 函数检查光线是否与球体相交。如果相交，使用 `where_Hit` 函数计算交点位置。然后，比较交点距离，更新最近的交点信息。

处理未相交情况

```
1 if (!closestSphere) {  
2     return Vector3(135, 206, 255); // 背景颜色  
3 }
```

如果没有找到任何交点，返回背景颜色（这里是一个预设的颜色值）。

计算光照

```
1 bool isReflection = (depth > 0);  
2 Vector3 directLight = calculatePhongLighting(hitPoint, normal, light, camera,  
    *closestSphere, spheres, isReflection);
```

计算直接光照效果。这里使用 `calculatePhongLighting` 函数，它将根据 Phong 光照模型计算光照效果。`isReflection` 标志用于指示是否需要考虑反射光线的影响。

处理反射

```
1 if (closestSphere->materialType == FABRIC && depth == 0) {  
2     return directLight; // 地面球不递归反射（避免倒影错误阴影）  
3 }  
4  
5 Vector3 reflectDir = ray.B - normal * 2.0 * (ray.B * normal);  
6 Ray reflectRay(hitPoint + normal * 1e-4, reflectDir, 0);  
7 Vector3 reflectedLight = traceRay(reflectRay, spheres, light, depth + 1) *  
    closestSphere->reflectivity;
```

如果材质允许反射（非布料材质或递归深度大于 0），计算反射光线的方向，并递归调用 `traceRay` 函数处理反射光线。这里还考虑了材质的反射率。

材质特定处理

```
1 if (closestSphere->materialType == FABRIC) {  
2     reflectedLight = reflectedLight * 0.05;  
3 }
```

对于特定材质（如布料），可能需要调整反射光线的颜色或强度。

返回最终颜色

```
1 return directLight + reflectedLight;
```

将直接光照和反射光照相加，得到最终的像素颜色。

4.5 phong 模型的引入

由于本人物理知识有限，光学尤甚，但由于材质的实现涉及大量光学知识，为此我在 CSDN 求助，了解到可以直接应用 phong 光照模型来模拟物体表面在光照下的反射特性。通过该模型调节环境光 (Ambient)、漫反射 (Diffuse) 和镜面反射 (Specular) 三种光照参数，即可实现对不同材质的模拟，十分简易，故采用之（具体见 Vector3 calculatePhongLighting 函数）。

4.6 窗口的绘制及 UI 设计

窗口绘制主要通过 ImageWidget 类实现。该类负责生成渲染结果并将其绘制到一个窗口部件上。其中，UI 设计主要借助于 Qt Designer 实现。

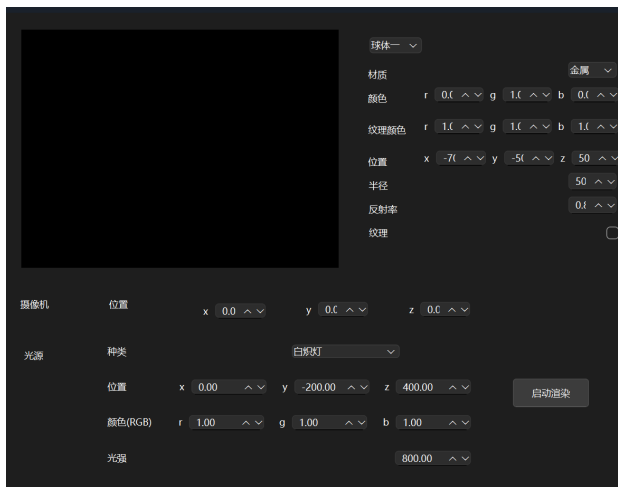


图 4.6.1

5 单元测试

效果展示

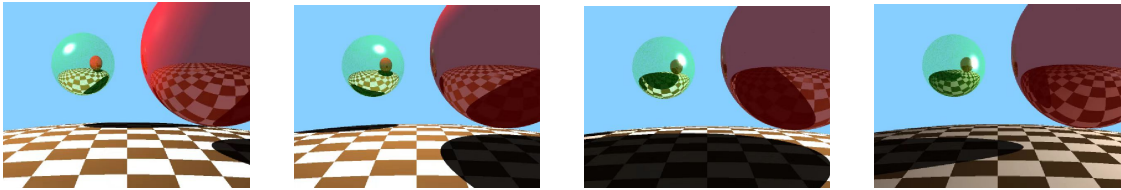


图 5.0.1: 光源位置变化

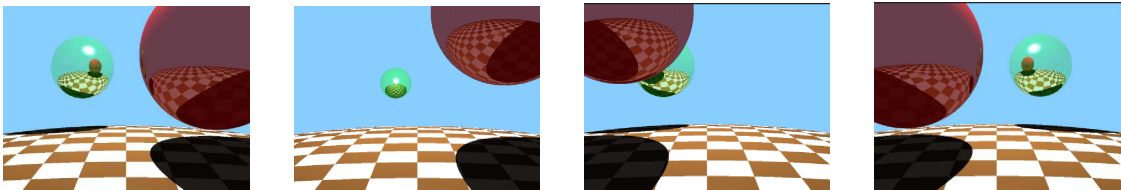


图 5.0.2: 球体位置修改

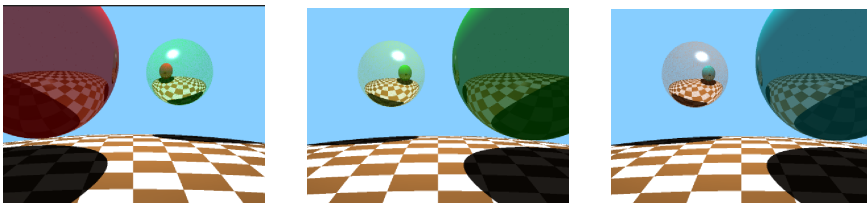


图 5.0.3: 球体颜色修改

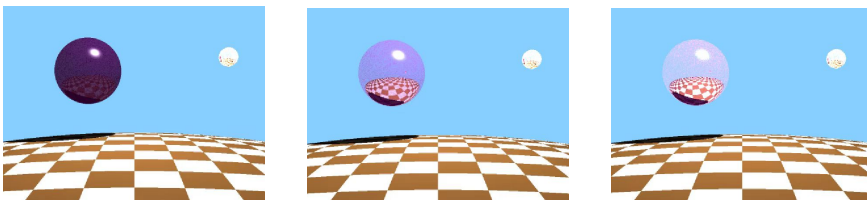


图 5.0.4: 球体反射率修改

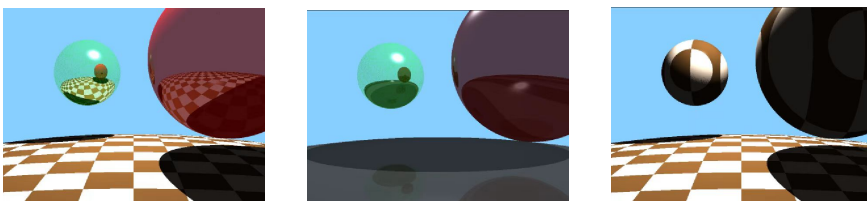


图 5.0.5: 球体材质修改

项目概述

该项目允许用户通过修改不同的参数来观察渲染效果的变化。更多参数调节效果，用户可以通过以下链接访问项目的详细内容和源代码：

- 项目视频介绍：<https://www.bilibili.com/video/BV1D2XpYsEiC/>
- 项目源代码：<https://github.com/Aurora1094/Ray-Tracing>

6 收获

对计算机图形学有了初步认识，了解了光线追踪的基本原理，感受到封装类的重要性。更重要的是，体验了在没有完全相同案例可供参考的情况下，独立查找资料解决问题的流程，我认为这对将来进一步的学习，乃至科研有着重要意义。最后，感谢身边所有人的支持、帮助和陪伴！